

Egyptian E-Learning University
Faculty of Computers & Information Technology

OLMEG CONNECT

Smart Marketplace Platform Using Flutter & Firebase

Supervised by
DR. ASMAA HEKAL
Assistant Eng. MOHAMED SAIED

[ALEX] - 2026

By

Name: Youssef Hafez El Morsy ID 2200028

Name: Merna Mohamed Naeim ID 2201804

Name: Mina Hany Samir ID 2202185

Name: Youssef Abdelmonem Aly ID 2201393

Name: Rayan Ahmed ID 2201792

Name: Abdelrahman Anwar ID 2101962

Name: Mahmoud Haroun ID 2100914

Name: Ehab Ibrahim ID 2100611

Name: Adham Ahmed Hamdy ID 2100309

Name: tarek Mohamed abd elhakim ID 2101660

Acknowledgments

We would like to express our sincere appreciation to Dr. Asmaa Hekal for her continuous guidance, supervision, and valuable feedback throughout all stages of this project. Her support helped improve the project idea, documentation structure, and final presentation.

We also extend our thanks to Eng. Mohamed Saied for his technical assistance and helpful recommendations during the development and documentation phases. Special thanks go to the Faculty of Computers & Information Technology at the Egyptian E-Learning University for providing the academic environment and resources that supported the completion of this project.

We appreciate all team members who contributed to design, development, testing, debugging, documentation, screenshots, and project demonstration.

Contents

Abstract

Chapter 1 - Introduction

Chapter 2 - Literature Review / Related Work

Chapter 3 - Proposed System

Chapter 4 - Implementation

Chapter 5 - Testing & Evaluation

Chapter 6 - Results & Discussion

Chapter 7 - Conclusion & Future Work

References

Appendix A - Application Screenshots

Appendix B - System Diagrams

Appendix C - Future Links and Deployment

Abstract

The rapid growth of e-commerce and mobile marketplace applications has increased the demand for systems that support buying and selling products in an easy, flexible, secure, and cost-effective way. Many existing marketplace applications focus on a limited product type, such as used products only, handmade products only, or business-to-consumer new products. This separation forces users to rely on several different platforms and creates a fragmented buying and selling experience.

Olmeg Connect is a Flutter and Firebase smart marketplace platform designed to provide a unified marketplace for new products, used products, and handmade products. The application supports product publishing, marketplace browsing, product details, favorites, advanced search, seller communication, social feed posts, ratings, reviews, reports, notifications, and role-based admin tools.

The application is developed using Flutter to support cross-platform deployment and a consistent responsive interface. Firebase is used as the backend solution for authentication, Firestore database operations, media storage, messaging, hosting, and future cloud functions. Riverpod is used for state management and GoRouter is used for navigation and authentication redirects. The project follows a feature-first modular structure, making it scalable and maintainable.

The current version demonstrates a strong prototype/product foundation with UI/UX progress of 90%, backend progress of 80%, chat progress of 80%, AI moderation progress of 60%, and payment progress of 40%. Future work includes integrating real AI moderation APIs, completing the payment gateway flow, improving Firebase security rules, adding delivery tracking, and preparing the app for Google Play deployment.

Chapter 1 - Introduction

1.1 Background

Mobile marketplace applications have become an essential part of modern digital commerce. They allow users to buy, sell, search, and communicate through mobile devices at any time. However, users often need different applications for different product types: classified apps for used products, e-commerce platforms for new products, and specialized platforms for handmade items.

Olmeg Connect was created to combine these experiences into one application. It supports a hybrid marketplace model that includes new products, used products, handmade products, seller profiles, chat, social content, notifications, and admin moderation.

1.2 Problem Statement

- Existing platforms often separate new, used, and handmade products into different systems.
- Many platforms have weak moderation and unsafe listings.
- Communication between buyers and sellers is often not integrated effectively.
- Some systems lack strong admin dashboards and reports management.
- Users need a modern mobile UI with search, favorites, notifications, and localization.

1.3 Project Objectives

- Build a scalable smart marketplace application.
- Support buying and selling new, used, and handmade products.
- Implement secure authentication with Firebase Auth.
- Store application data using Cloud Firestore.
- Store product and profile images using Firebase Storage.
- Enable real-time buyer-seller communication through chat.
- Provide advanced product search and filtering.
- Add social feed posts and user interaction.
- Provide admin moderation tools for safety and platform quality.
- Support notifications through Firebase Cloud Messaging.
- Support English and Arabic localization.
- Prepare the system for future payment, AI moderation, and deployment improvements.

1.4 Scope

The scope of Olmeg Connect includes mobile/web marketplace functionality, Firebase backend integration, product publishing, searching, browsing, user profiles, favorites, social posts, chat, notifications, ratings, reviews, reports, seller tools, and admin-only access pages. The project also includes placeholders for APK download, Figma design, and Firebase console screenshots that can be added later.

Chapter 2 - Literature Review / Related Work

Several online marketplace platforms exist, but most of them focus on one business model or product category. OLX is widely used for second-hand product exchange, while Etsy focuses mainly on handmade and creative products. Amazon and Noon focus mainly on new products and large-scale e-commerce. Facebook Marketplace supports local exchange but does not provide the same structured seller tools, moderation workflow, or dedicated marketplace architecture.

Olmeg Connect addresses this gap by combining new products, used products, handmade products, social feed interaction, buyer-seller chat, admin moderation, notifications, and seller tools inside one Flutter application.

Platform	Strength	Limitation Compared with Olmeg Connect
OLX	Strong local used-product marketplace	Mainly focused on classified/used listings; limited social/feed and admin tooling.
Facebook Marketplace	Easy social access and local selling	Less structured for product moderation, seller dashboards, and dedicated marketplace flows.
Amazon / Noon	Large e-commerce infrastructure	Mainly focused on new products and formal sellers; less flexible for peer-to-peer used items.
Etsy	Strong handmade marketplace	Does not combine used, new, and local marketplace categories in the same way.
Olmeg Connect	Combines marketplace, social feed, chat, admin tools, localization, Firebase backend	Payment and real AI moderation require future completion.

Chapter 3 - Proposed System

3.1 Proposed Solution

Olmeg Connect is proposed as a hybrid marketplace system that supports multiple product types and multiple user roles. The system includes buyers, sellers/merchants, and administrators. Buyers can browse, search, favorite, chat, order, review, and report. Sellers can publish products, manage inventory, communicate with buyers, and access seller-related tools. Admins can access admin-only pages for moderation, notifications, reports, and product review.

3.2 Technology Stack

Category	Technology / Package	Purpose
Frontend	Flutter	Cross-platform application development.
Language	Dart >=3.0.0 <4.0.0	Programming language used by Flutter.
State Management	flutter_riverpod, state_notifier	Scalable state management and provider-based architecture.
Navigation	go_router	Route management and authentication redirects.
Backend	Firebase Authentication	Login, registration, and session handling.
Database	Cloud Firestore	Stores users, products, chats, posts, orders, notifications, and reports.
Storage	Firebase Storage	Stores product images, avatar images, and post images.
Notifications	Firebase Messaging	Push notification and device token support.
Payments	flutter_paypal	PayPal integration setup and future checkout flow.
Images	image_picker, cached_network_image	Image selection and optimized network image loading.
UI Helpers	shimmer, gap, flutter_rating_bar	Loading skeletons, spacing, and ratings UI.
Utilities	uuid, intl, timeago, shared_preferences	IDs, formatting, timestamps, and local preferences.

3.3 Architecture

The project uses a feature-first structure under lib/features. Many features follow a layered architecture with data, domain, and presentation layers. Shared code lives under lib/core and includes constants, router, services, theme, widgets, localization, and utilities. This structure resembles clean architecture and makes the project easier to scale.

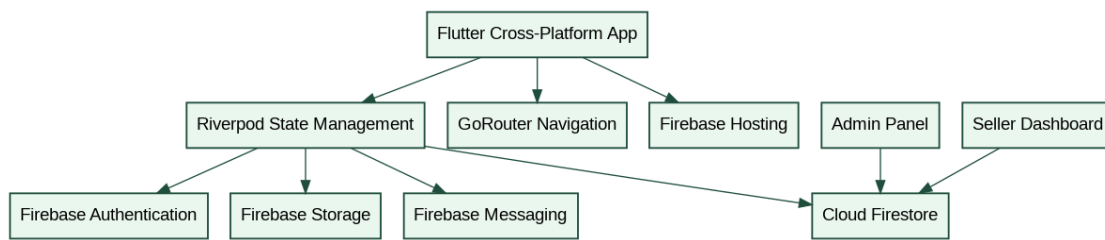


Figure 3.1 - System Architecture Diagram

3.4 Project Structure

olmeg_connect/

|- lib/

| |- core/

| | |- constants/

| | |- errors/

| | |- localization/

| | |- router/

| | |- services/

| | |- theme/

| | |- utils/

| | `-- widgets/

| `-- features/

| |- admin/

| |- analytics/

| |- auth/

| |- cart/

| |- chat/

| |- home/

| |- notifications/

| |- payments/

- | |- posts/
- | |- products/
- | |- profile/
- | |- ratings/
- | |- search/
- | |- settings/
- | `- shell/
- |- test/
- |- web/
- |- android/
- |- ios/
- |- macos/
- |- linux/
- |- windows/
- |- functions/
- |- firebase.json
- |- firestore.rules
- |- storage.rules
- `- pubspec.yaml

3.5 Navigation Structure

- /login
- /register
- /home
- /product/:id
- /cart
- /create-post
- /demo
- /chats
- /chat/:id
- /settings
- /favorites
- /my-products
- /privacy-policy
- /terms
- /edit-profile
- /search
- /advanced-search

- /notifications
- /admin
- /admin/notify
- /admin/moderation

Unauthenticated users are redirected to /login. Authenticated users trying to access /login or /register are redirected to /home. The main shell uses an indexed bottom navigation layout for Home, Feed, Add Product, Chat, Notifications, and Profile.

3.6 Firebase Collections

Collection	Purpose
users	Stores user profiles, email, name, role, admin flag, account type, merchant status, and FCM token.
products	Stores approved product listings.
product_submissions	Stores products waiting for admin moderation.
categories / subcategories	Stores category hierarchy and product grouping.
carts	Stores persisted cart records.
orders	Stores order records, buyer, sellers, items, address, totals, and payment summary.
payments	Stores payment records and payment method/status.
chats	Stores buyer-seller product conversations and messages.
posts	Stores social marketplace posts.
posts/{postId}/comments	Stores comments on posts.
favorites	Stores favorite products.
reviews	Stores product reviews.
ratings	Stores seller/user ratings.
reports	Stores reports against products, sellers, reviews, or posts.
notifications	Stores user notifications.
admin_notifications	Stores admin broadcast and notification records.
merchant_verifications	Stores merchant approval requests.
sellerProfiles	Stores seller profile data.
analytics_events	Stores tracked product/user events.



Figure 3.2 - Entity Relationship Diagram



Figure 3.3 - Use Case Diagram

Chapter 4 - Implementation

4.1 Main Application Flow

Application entry starts in `lib/main.dart`. The startup process ensures Flutter bindings are initialized, initializes Firebase using `lib/firebase_options.dart`, starts Riverpod ProviderScope, initializes notification services after the first frame, and loads router, theme mode, and locale providers. The router file is `lib/core/router/app_router.dart`.

4.2 Authentication

The authentication feature includes login, register, Firebase-backed auth state handling, user roles, merchant account type support, and admin detection. The main screens are `login_screen.dart` and `register_screen.dart`. The login UI supports English and Arabic, and the register UI supports merchant registration options.

4.3 Products and Marketplace

The products feature includes product listing, product details, add product flow, category filtering, product cards, badges, gallery widgets, seller UI components, favorites, my products, product reviews, ratings, variants, stock, return policy, delivery estimate, and moderation status. Important screens include `shop_screen.dart`, `add_product_screen.dart`, `product_detail_screen.dart`, `cart_screen.dart`, and `home_screen.dart`.

4.4 Search

The search feature contains a search screen, advanced search screen, search provider, repository, datasource layers, and filter entity support. The advanced search screenshot shows filters for keyword, category, condition, and price range.

4.5 Cart, Orders, and Payments

The app contains cart screen and cart provider logic. Checkout review and order detail screens support order creation and payment summaries. Payment records are stored in Firestore payments collection using `paymentMethod: paypal`. The `flutter_paypal` package exists in the dependencies, but a full visible PayPal checkout UI flow is not complete yet.

4.6 Social Feed and Posts

The social feed includes feed screen, create post screen, post profile screen, product matching utilities, image grid widgets, preview widgets, reaction bar, and enhanced post card widgets. This creates a social-commerce layer where users can share marketplace-related posts and interact with community content.

4.7 Chat

The chat feature includes chat list screen, chat detail screen, chat provider, entity, model, and datasource layers. Chat is linked to products so buyers and sellers can communicate about specific listings. Current chat progress is 80%.

4.8 Notifications

Notifications include a notifications screen, notification provider and repository layers, Firebase Cloud Messaging integration, and foreground/background notification handling service. User FCM tokens are stored in user documents.

4.9 Admin Module

The admin module contains admin dashboard, admin notification screen, product moderation screen, reports management, and merchant verification features. The admin-only profile/access page provides access to Admin Dashboard, Admin Notifications, Product Moderation, Help & Support, account management, and sign out. This page is visible only to admin users and supports role-based UI visibility using the user role/admin flag.

4.10 Ratings, Reviews, Reports, Profile, and Settings

The ratings and reviews modules include rating entity, model, datasource, repository, provider, and rating widget. Reports support reporting products, sellers, reviews, or posts. Profile and settings include edit profile, favorites, privacy policy, terms, app settings, dark mode, and language switching.

4.11 AI Moderation

The moderation service is implemented in `product_moderation_service.dart`. The current implementation is rule-based local moderation, not an external AI API. It checks title

length, description length, price validity, and blocked terms such as weapons, drugs, fake IDs, stolen/counterfeit items, hacking, and explosives. Approved products go directly to products, while rejected or suspicious products go to product_submissions for admin review.

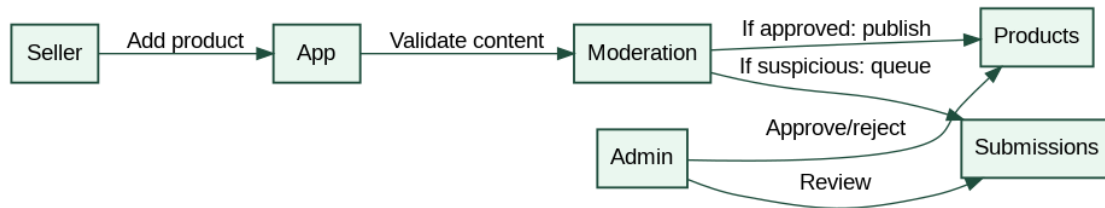


Figure 4.1 - Product Moderation Sequence

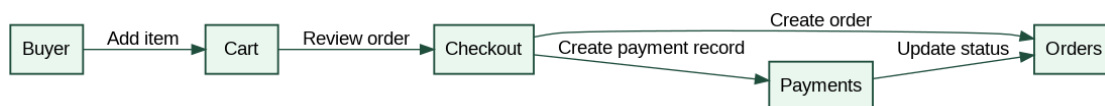


Figure 4.2 - Checkout Sequence

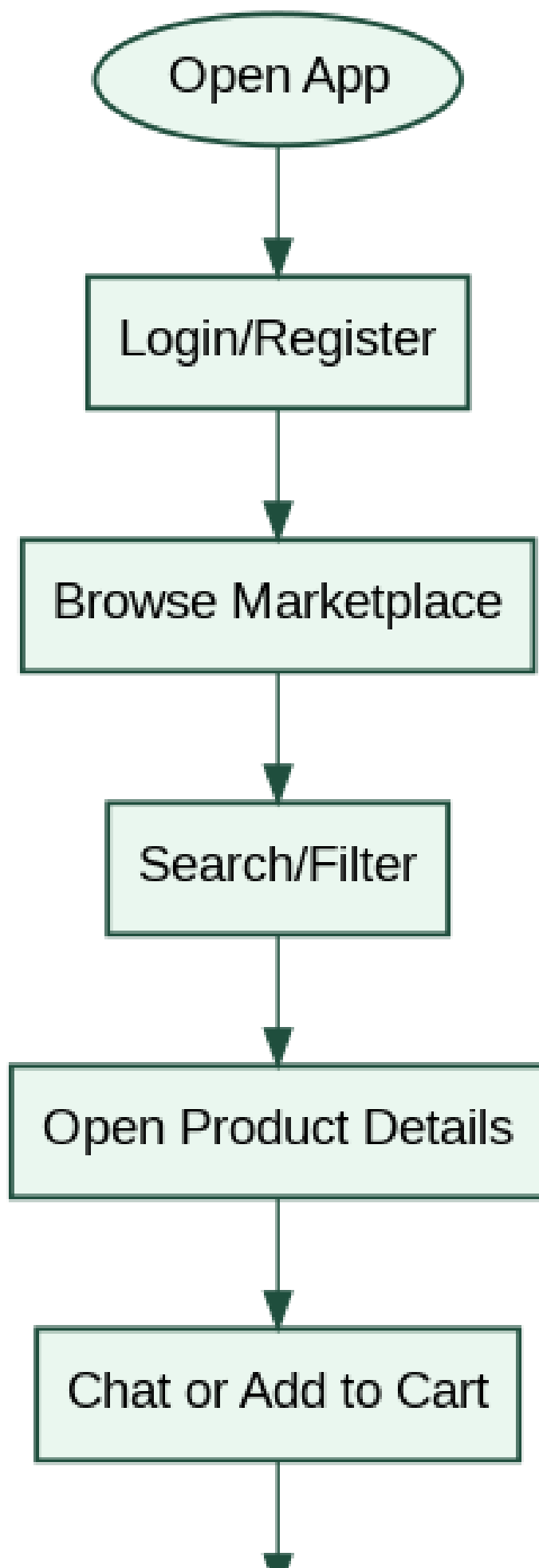


Figure 4.3 - Main User Activity Flow

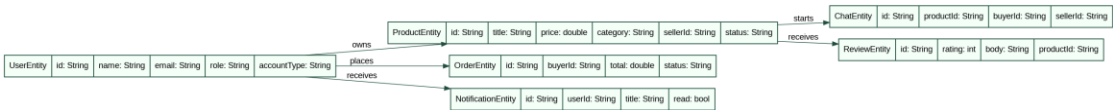


Figure 4.4 - Class Diagram

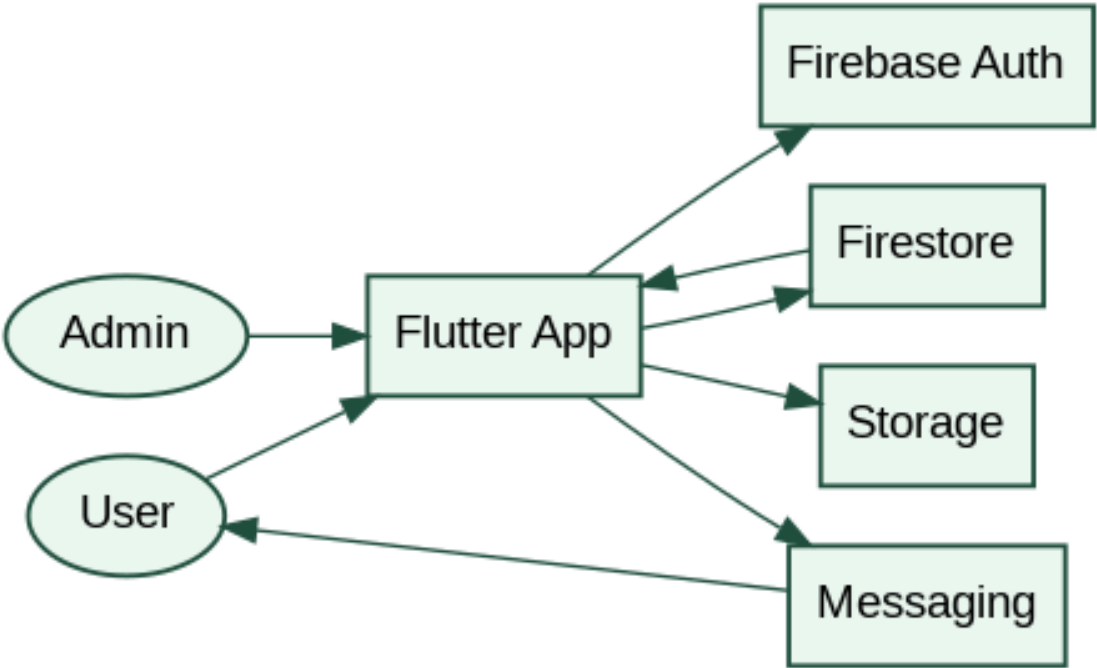


Figure 4.5 - Data Flow Diagram

Chapter 5 - Testing & Evaluation

5.1 Testing Strategy

Testing was conducted through manual testing, widget smoke tests, cart calculation tests, order creation tests, merchant delivery tests, Firebase integration testing, responsive UI testing, localization testing, and navigation testing. The repository includes test files and supporting QA documents such as TESTING_REPORT.md, TEST_REPORT.md, FINAL_TEST_SUMMARY.md, and TESTING_CHECKLIST.md.

5.2 Current Progress

Area	Progress
UI/UX	90%
Backend	80%
Payments	40%

Chat	80%
AI Moderation	60%

5.3 Feature Status

Feature	Status / Notes
Authentication	Implemented with Firebase Auth and auth redirects.
Products	Implemented: listing, details, add product, categories, images, favorites.
Search	Implemented with search and advanced search screens.
Notifications	Implemented with Firebase Messaging and notification screen.
Reviews/Ratings	Implemented data layer and UI widgets.
Reports	Implemented for product/seller/review/post reporting.
Orders	Implemented order records, order details, and checkout review logic.
Admin moderation	Implemented admin moderation access and product moderation queue.
Seller dashboard	Implemented/partial seller tools for dashboard, inventory, and orders.
Social feed/posts	Implemented feed, create post, reactions, image posts.
Favorites	Implemented.
Web support	Configured using Firebase Hosting and Flutter web.
Payments	Partial: Firestore records and PayPal package; full visible checkout flow pending.
AI moderation	Partial: rule-based moderation; external AI API planned.

5.4 Challenges and Solutions

Challenge	Solution
Flutter Gradle build failures	Updated dependencies, cleaned cache, and aligned Flutter/Gradle configuration.
Firebase configuration conflicts	Reconfigured FlutterFire setup and Firebase platform files.
Flutter web image decoding issue	Used cached network image and fallback handling.
Navigation complexity	Used GoRouter for route protection and nested navigation.
State management scaling	Used Riverpod and feature-specific providers.
Firestore structure growth	Split data into feature-specific collections.
Notification setup issues	Added Firebase Messaging handlers and token storage.
Responsive overflow in product details	Identified in testing; solved using scrollable/adaptive layouts and SafeArea.

Chapter 6 - Results & Discussion

The current version of Olmeg Connect successfully demonstrates a multi-feature marketplace ecosystem. Compared with the initial template version that mentioned only a few screens, the current app includes many more production-level modules: authentication, marketplace browsing, advanced search, add product, product details, chat, notifications, social feed, admin access, settings, profile, localization, and role-based tools.

The screenshots provided show a consistent dark-mode UI, green accent branding, reusable card components, social-commerce integration, Arabic/English support, and a navigation shell structure. The project is stronger than a simple CRUD application because it combines marketplace, social media, chat, moderation, notifications, seller tools, analytics, localization, and admin systems.

6.1 Limitations

- Firebase rules are development-friendly and must be hardened before production.
- Payment integration is partial and requires a complete live checkout flow.
- AI moderation is currently rule-based and should later use an external AI API.
- Cloud Functions folder is configured but no full checked-in source implementation was found in the snapshot.
- Some UI overflow issues were identified and should be fixed before final demo.

Chapter 7 - Conclusion & Future Work

Olmeg Connect successfully demonstrates a scalable smart marketplace system built using Flutter and Firebase. The project combines product trading, social interaction, seller-buyer communication, notifications, moderation, admin tools, localization, and Firebase backend integration inside one application.

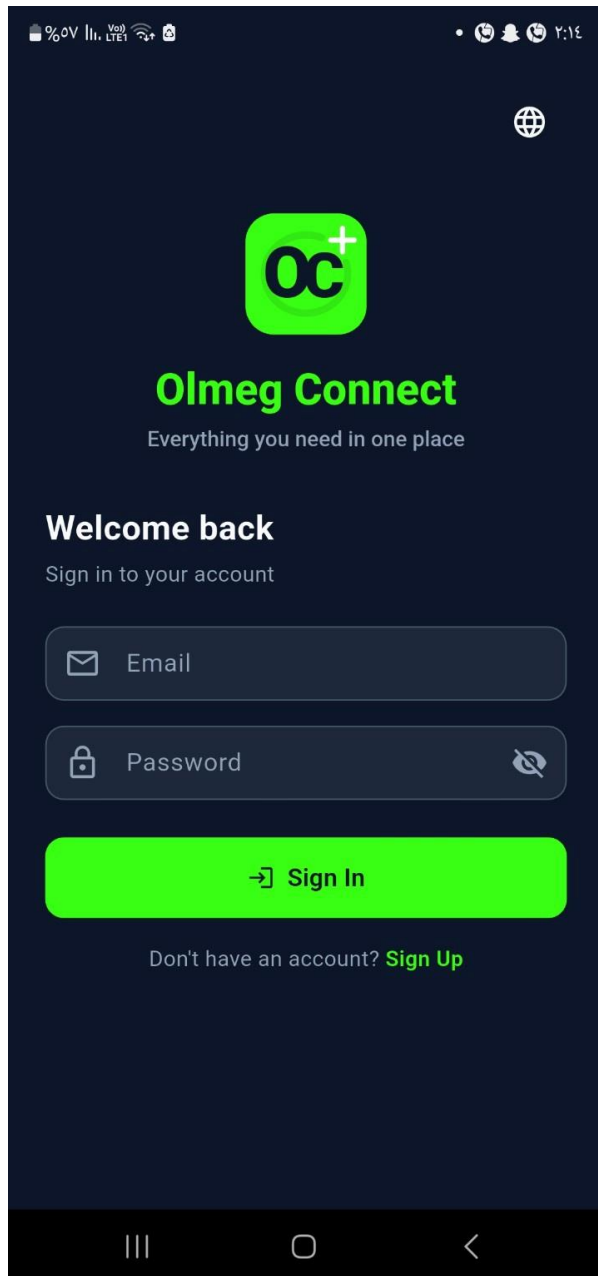
Flutter was chosen because it supports cross-platform development from a single codebase and provides responsive UI components. Firebase was selected because it provides ready-to-use authentication, database, storage, messaging, hosting, and future cloud backend capabilities. Riverpod and GoRouter improve scalability and maintainability by separating state and routing logic.

Future improvements include real AI moderation APIs, complete PayPal checkout, stronger security rules, delivery tracking, recommendation machine learning, production analytics, Google Play release, APK distribution, and improved Firebase Cloud

Functions.

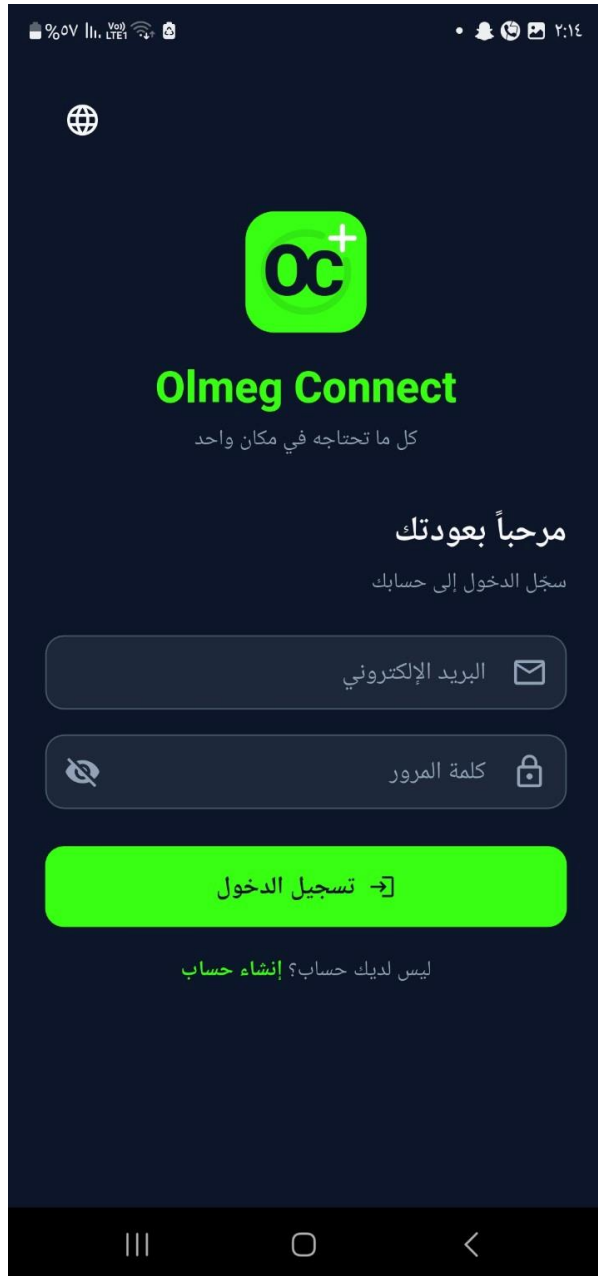
Appendix A - Application Screenshots

Figure A.1 - Login Screen - English Interface



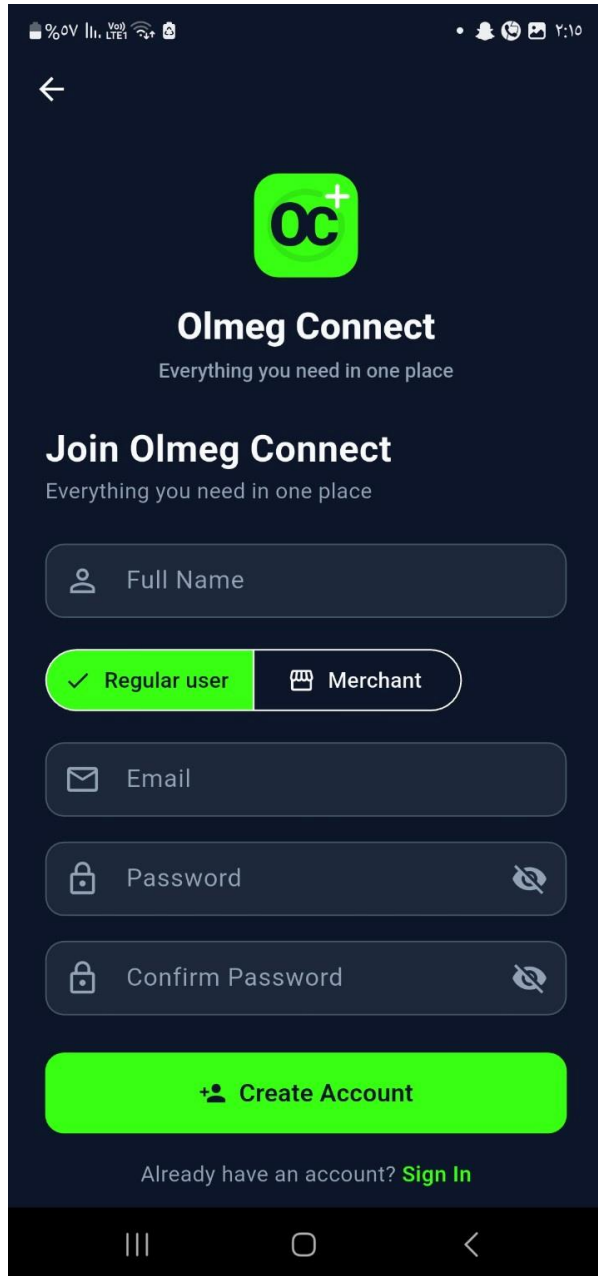
Login Screen - English Interface

Figure A.2 - Login Screen - Arabic Interface



Login Screen - Arabic Interface

Figure A.3 - Register Screen



The image shows a mobile application registration screen for 'Olmeg Connect'. The background is dark blue. At the top, there's a status bar with battery, signal, and time. Below it, a back arrow is in the top left. The app logo, 'OC+' in a green circle, is centered. Below the logo, the text 'Olmeg Connect' is in white, followed by the tagline 'Everything you need in one place'. A section titled 'Join Olmeg Connect' also has the same tagline. Below this, there are four input fields: 'Full Name' with a person icon, a role selector with 'Regular user' (checked) and 'Merchant' (unchecked), 'Email' with an envelope icon, and 'Password' and 'Confirm Password' both with lock icons and toggle visibility icons. A large green button with a plus and person icon is labeled 'Create Account'. At the bottom, a link says 'Already have an account? Sign In'. The bottom of the screen shows a black navigation bar with three icons: a hamburger menu, a square, and a back arrow.

0% 11:10 LTE1

←

OC+

Olmeg Connect
Everything you need in one place

Join Olmeg Connect
Everything you need in one place

Full Name

✓ Regular user Merchant

Email

Password

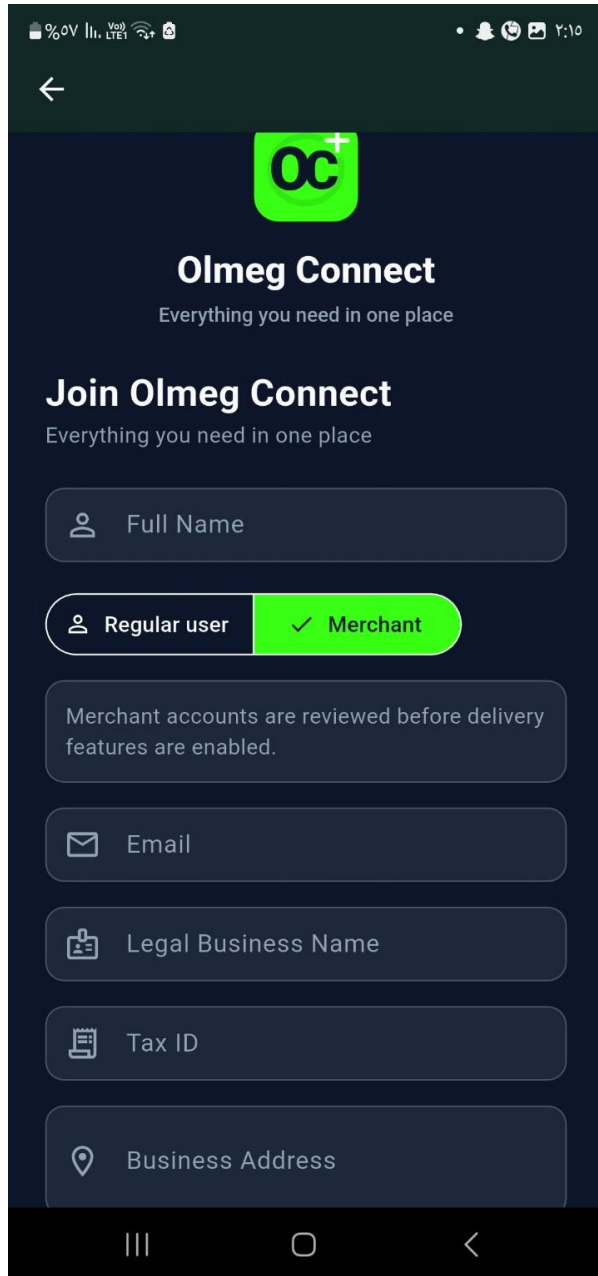
Confirm Password

+ Create Account

Already have an account? [Sign In](#)

Register Screen

Figure A.4 - Merchant Registration Screen



The image shows a mobile application interface for Olmeg Connect. At the top, there is a status bar with battery, signal, and time indicators. Below the status bar is a dark green header with a white back arrow. The main content area has a dark blue background. At the top of this area is the Olmeg Connect logo, which consists of a green square with the letters 'OC' and a plus sign. Below the logo is the text 'Olmeg Connect' and the tagline 'Everything you need in one place'. The main heading is 'Join Olmeg Connect', followed by the same tagline. There are five input fields: 'Full Name', 'Email', 'Legal Business Name', 'Tax ID', and 'Business Address'. Each field has a corresponding icon on the left. Below the 'Full Name' field, there are two buttons: 'Regular user' and 'Merchant'. The 'Merchant' button is highlighted in green and has a checkmark icon. Below these buttons, there is a text box that says 'Merchant accounts are reviewed before delivery features are enabled.' At the bottom of the screen is a dark green footer with three icons: a hamburger menu, a home button, and a back arrow.

Full Name

Regular user Merchant

Merchant accounts are reviewed before delivery features are enabled.

Email

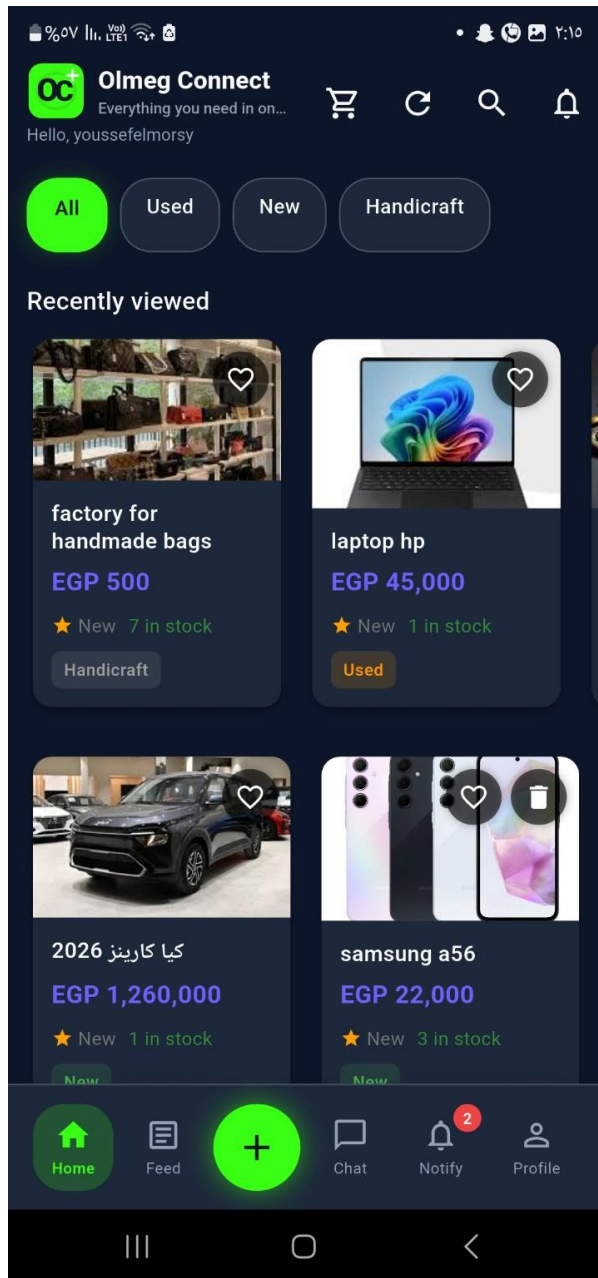
Legal Business Name

Tax ID

Business Address

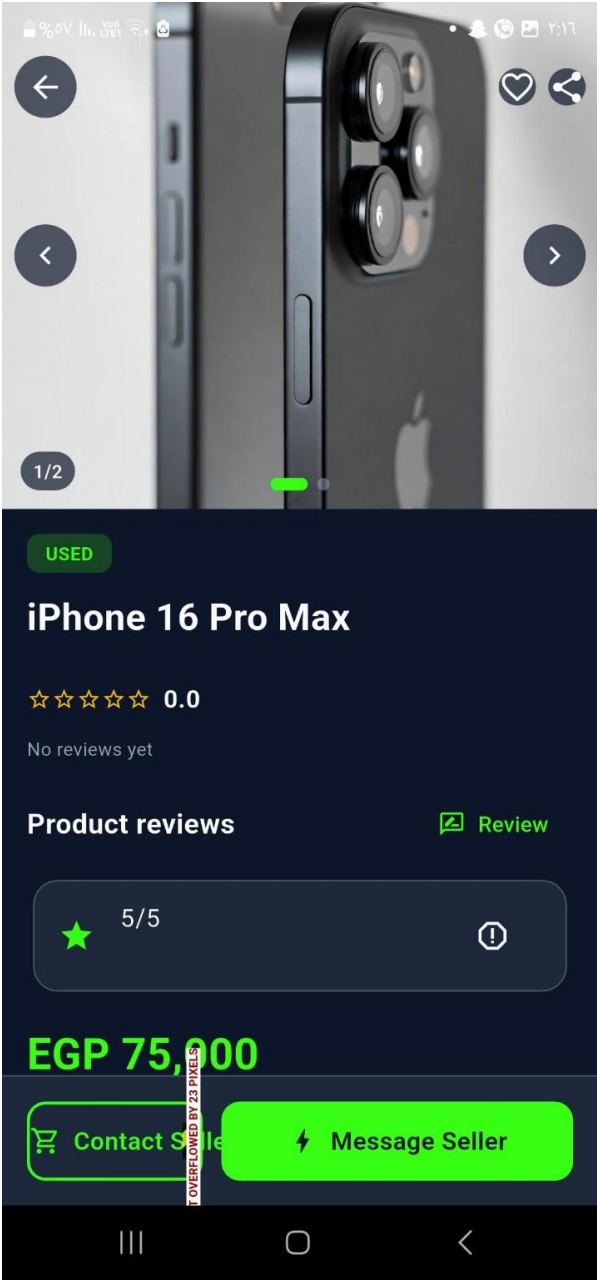
Merchant Registration Screen

Figure A.5 -



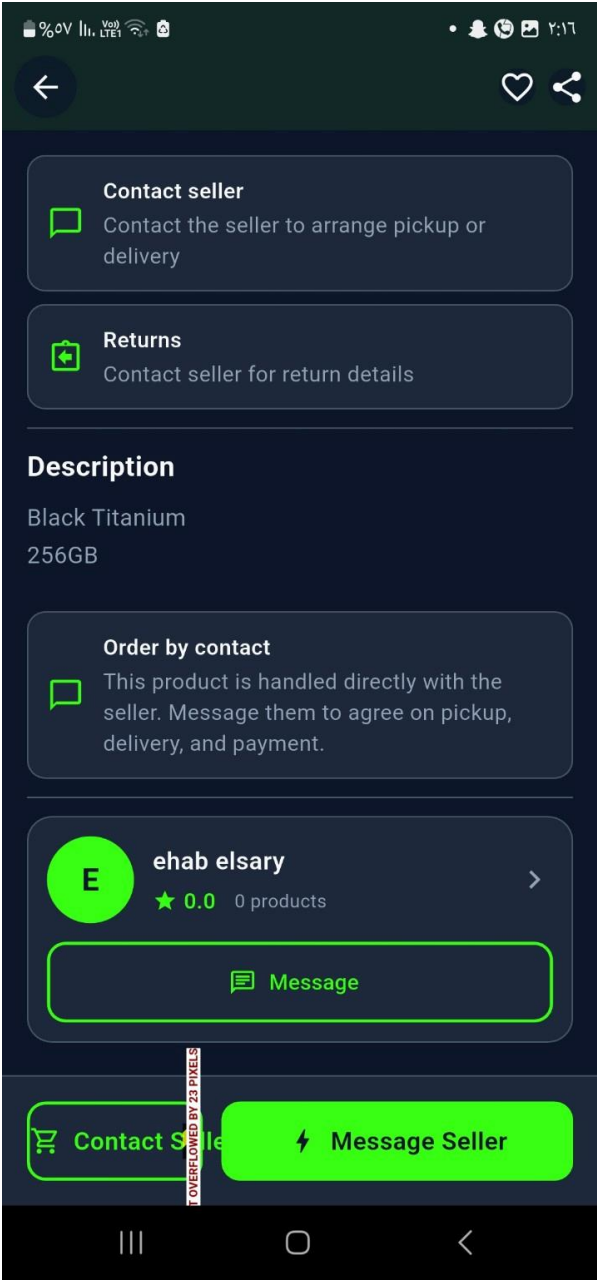
Settings / Profile Related Screen

Figure A.6 - Marketplace / Product Browsing Screen



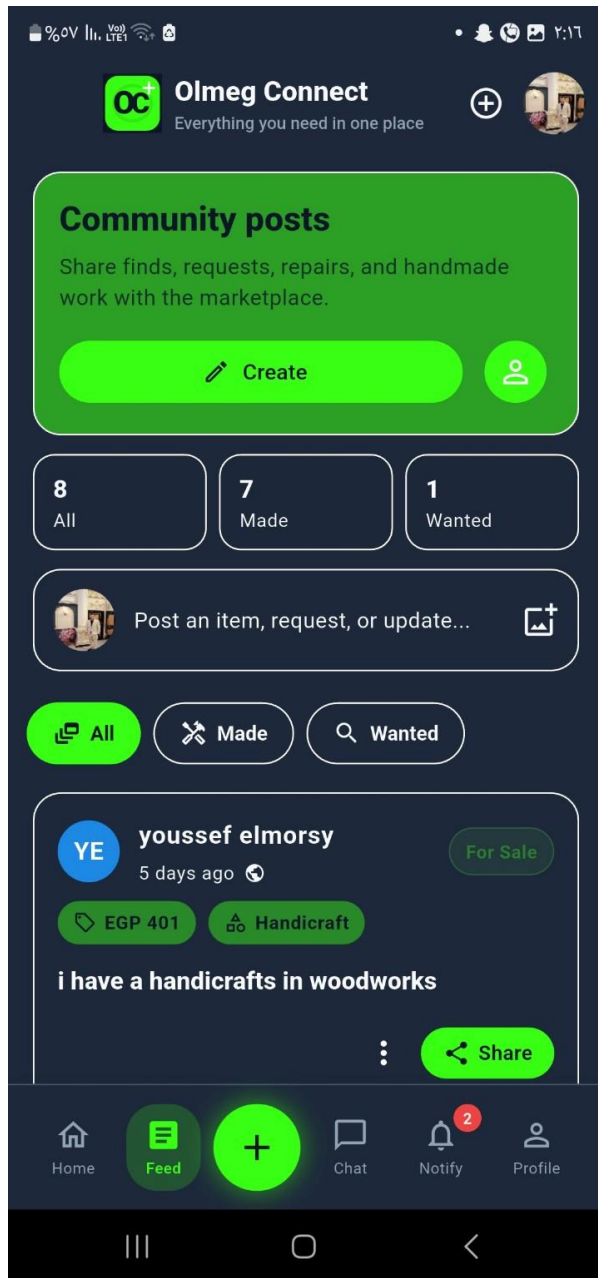
Marketplace / Product Browsing Screen

Figure A.7 - Product Details / Marketplace Interaction Screen



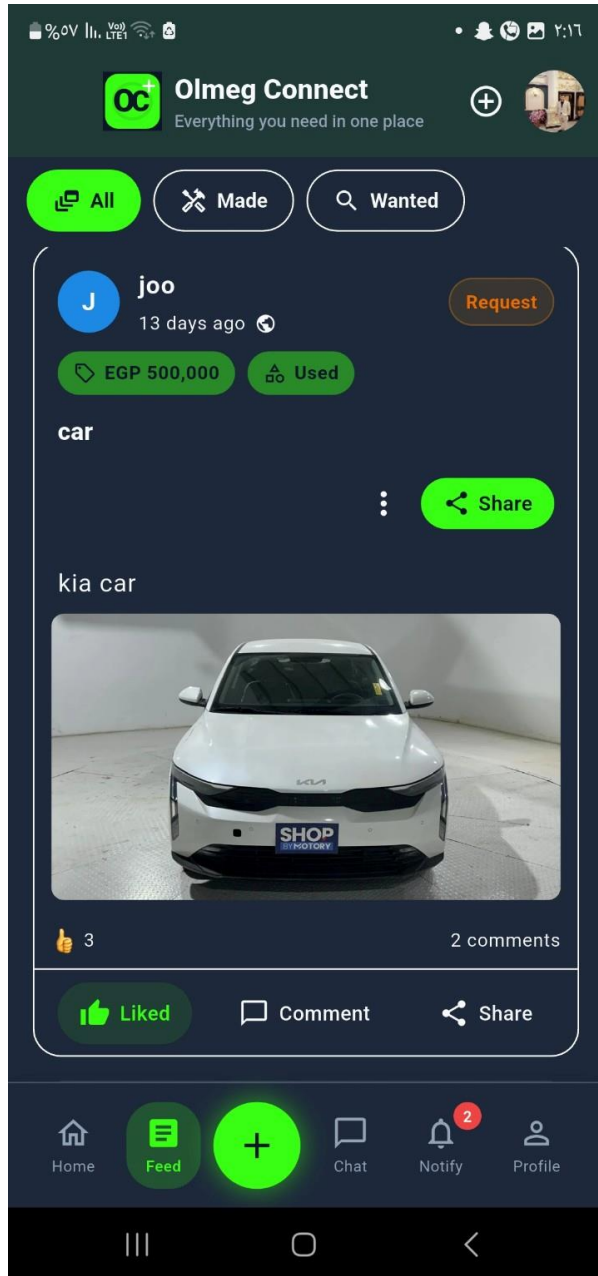
Product Details / Marketplace Interaction Screen

Figure A.8 - Advanced Search / Filter Screen



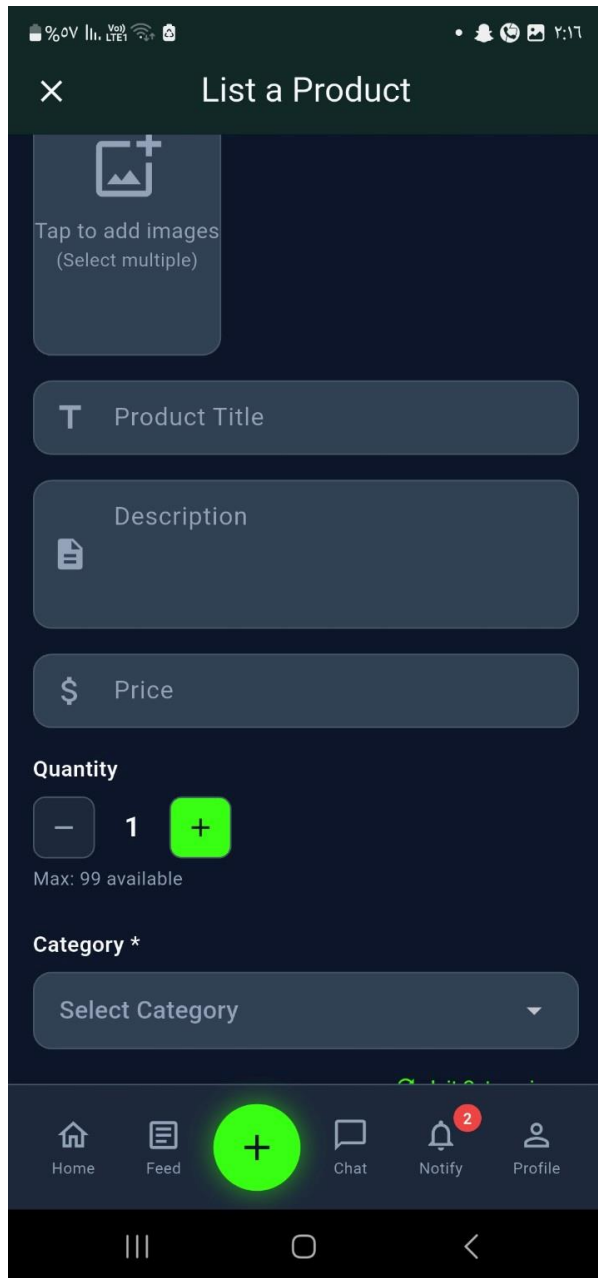
Advanced Search / Filter Screen

Figure A.9 - Notifications Screen



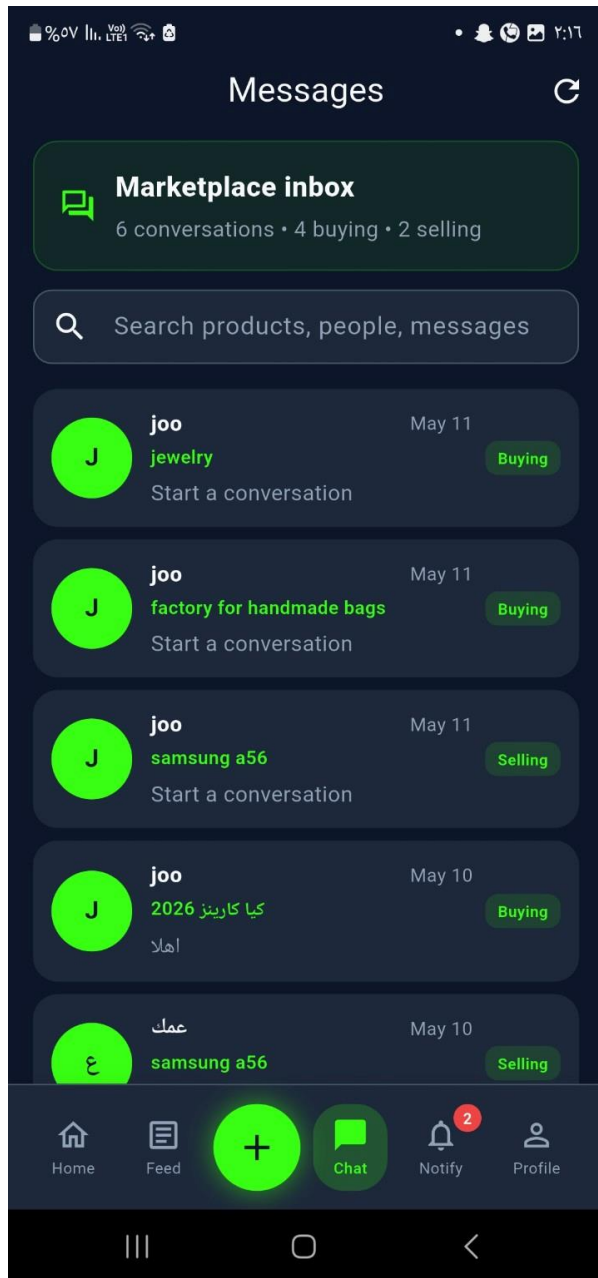
Notifications Screen

Figure A.10 - Profile Screen



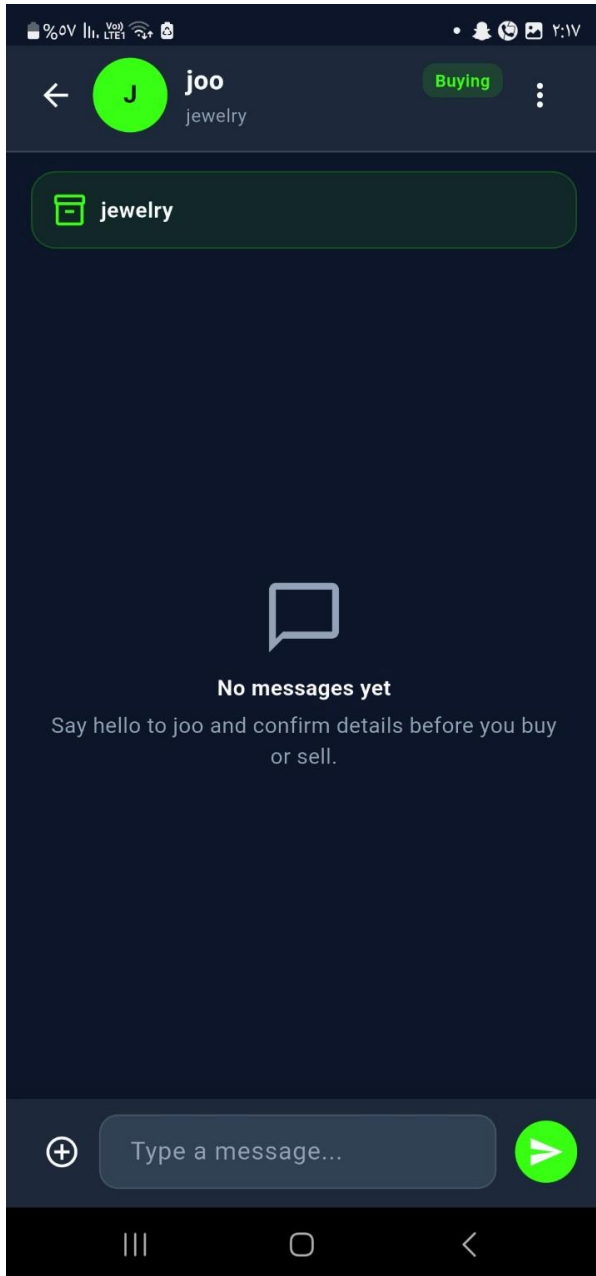
Profile Screen

Figure A.11 - Chat List Screen



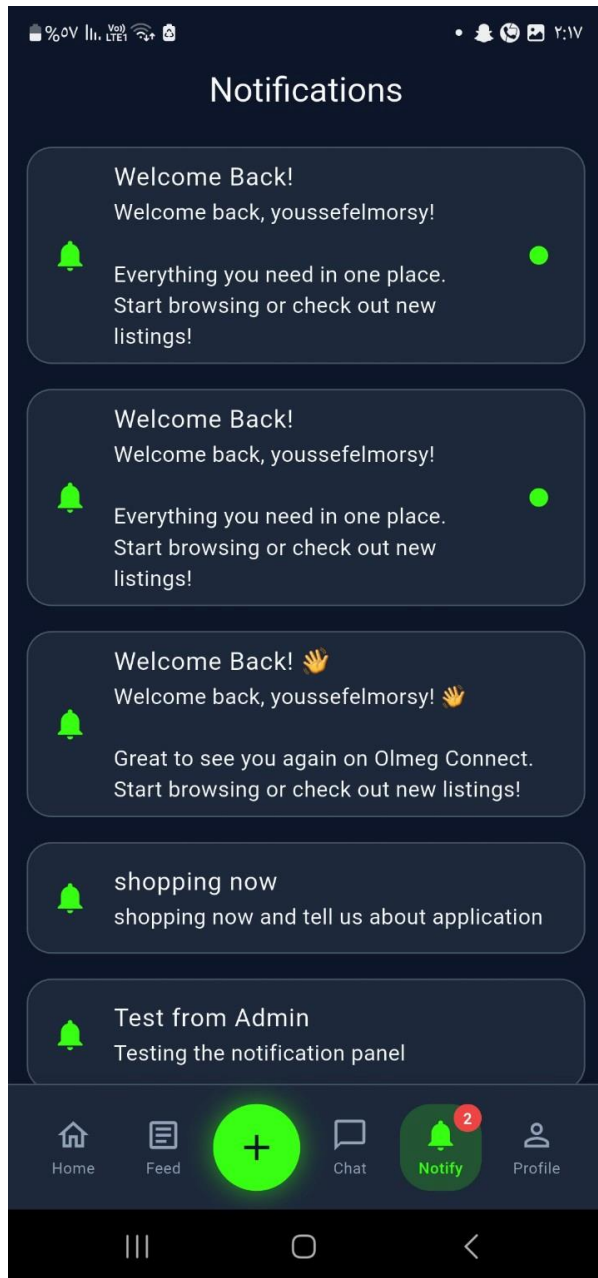
Chat List Screen

Figure A.12 - Chat Details Screen



Chat Details Screen

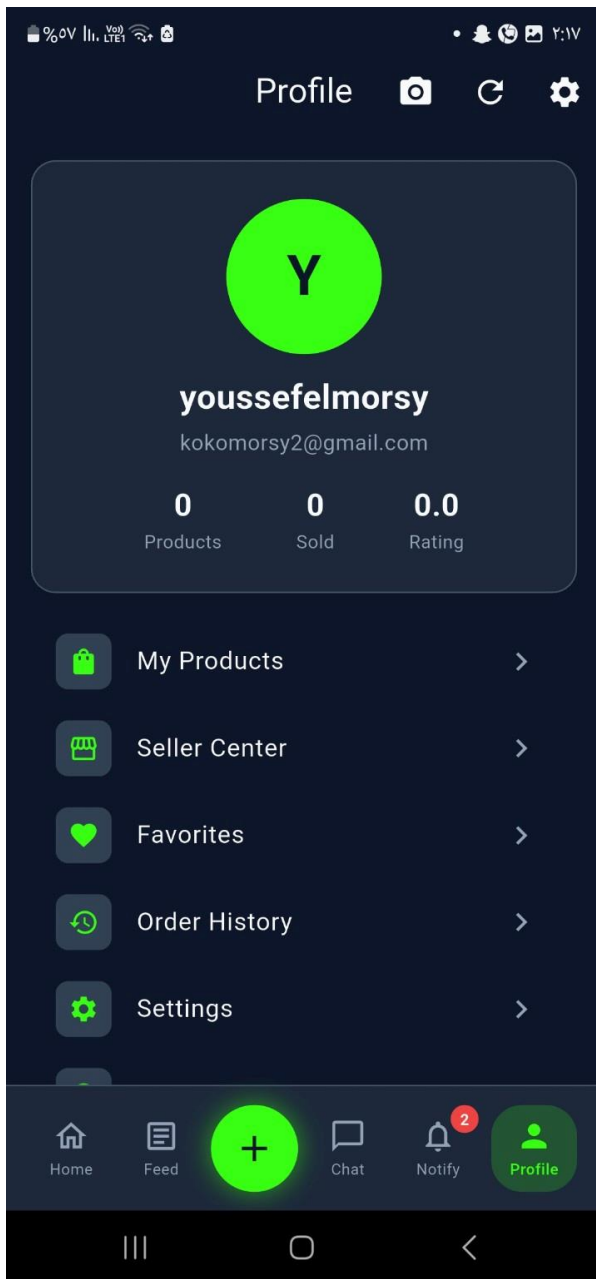
Figure A.13 – notification Screen



Notification Screen

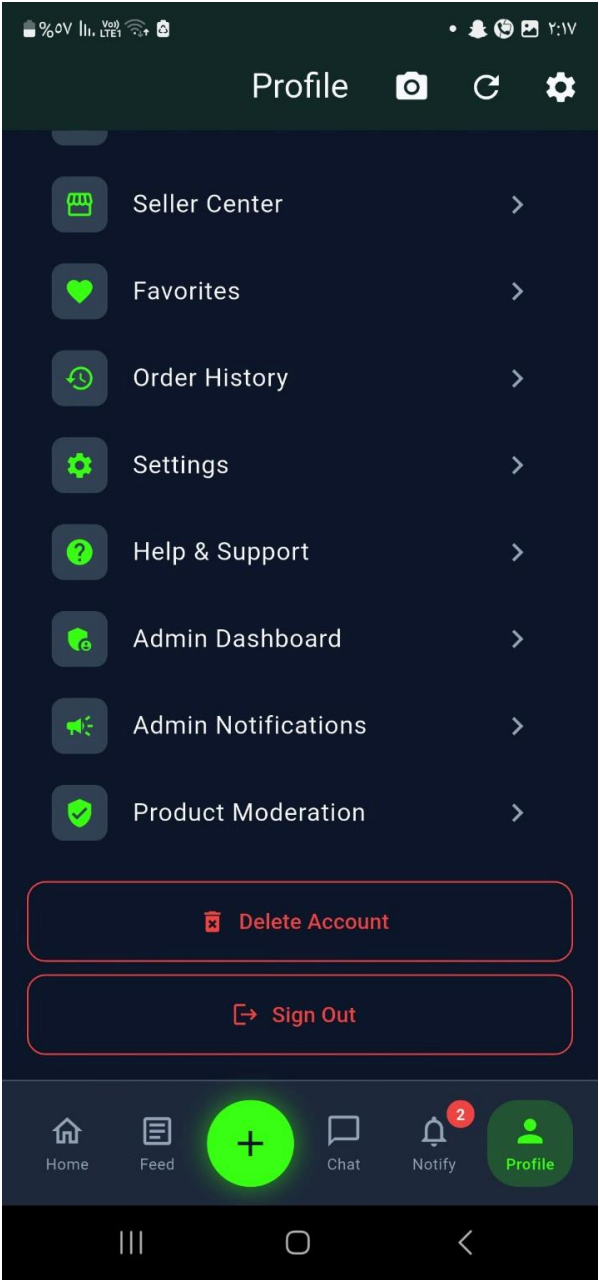
Figure A.14 - Settings / Profile Related Screen

Settings / Profile Related Screen



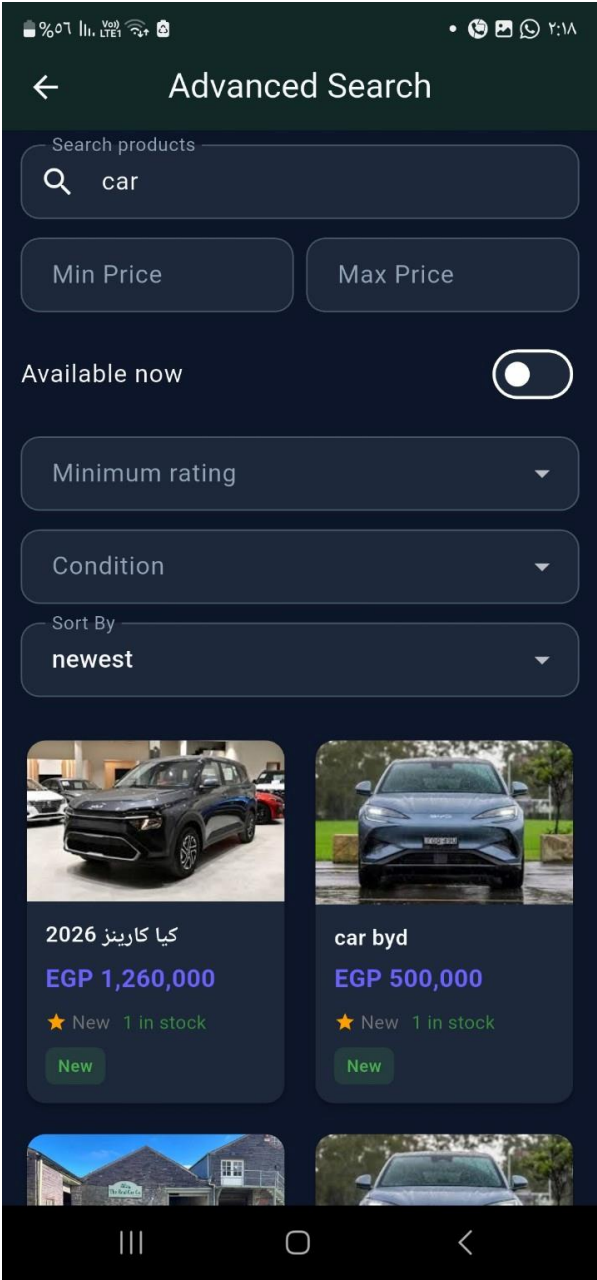
Settings / Profile Related Screen

Figure A.15 - Admin-Only Profile Access Panel



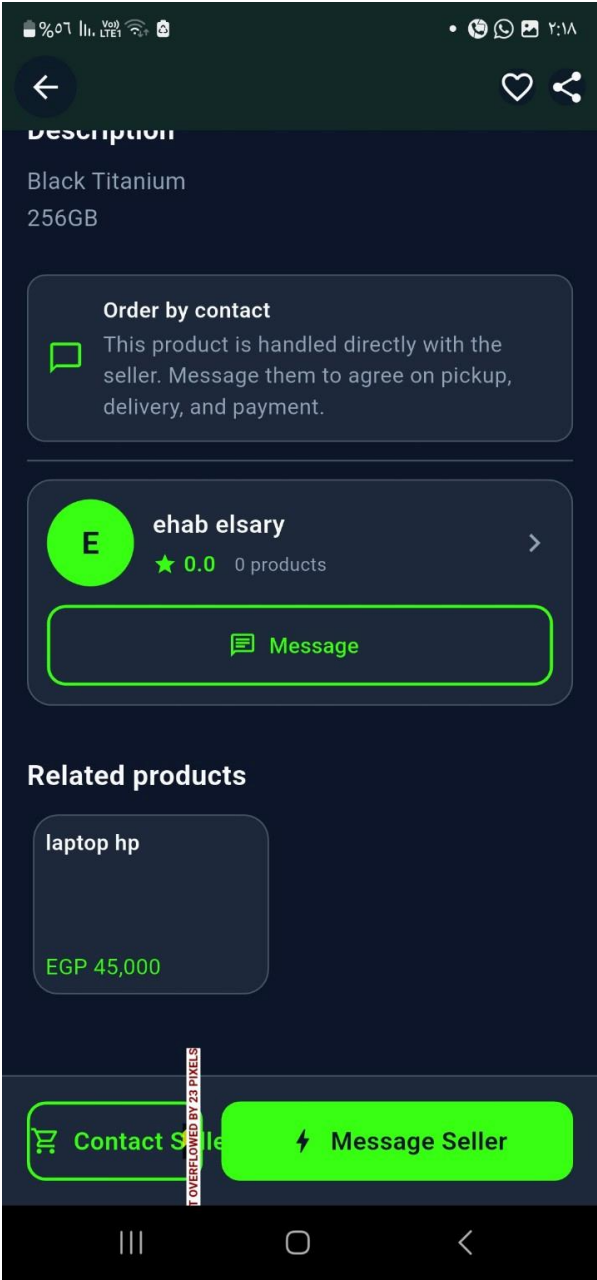
Admin-Only Profile Access Panel

Figure A.16 – advanced search



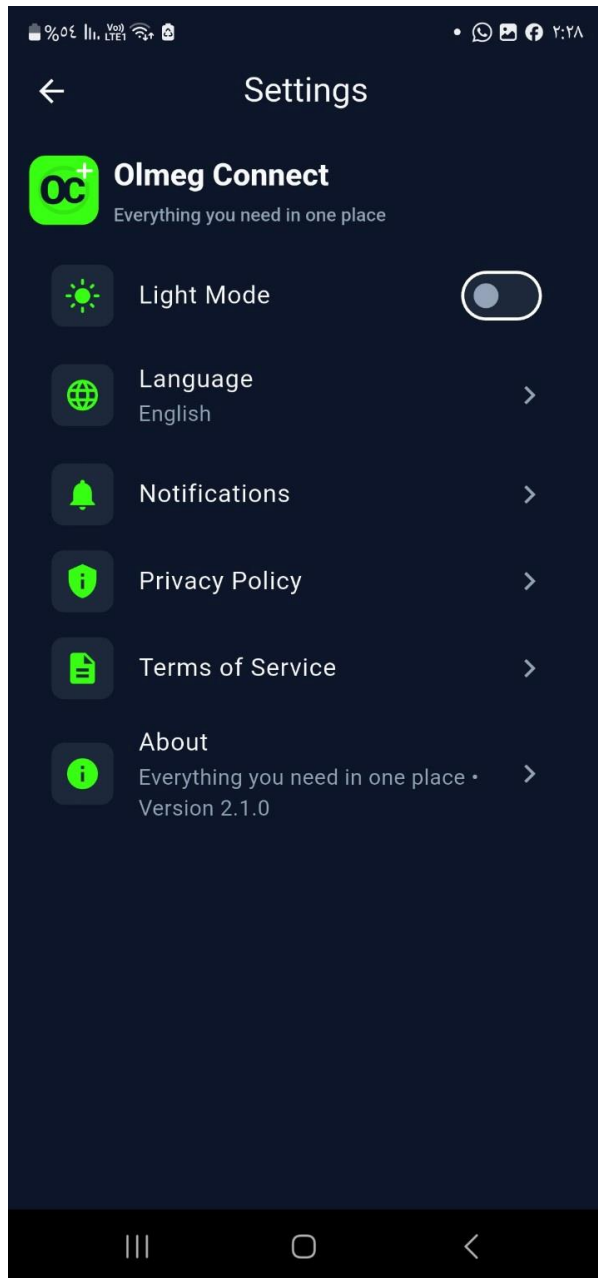
Advanced search

Figure A.17 - Product Details Screen With Responsive Test Observation



Product Details Screen With Responsive Test Observation

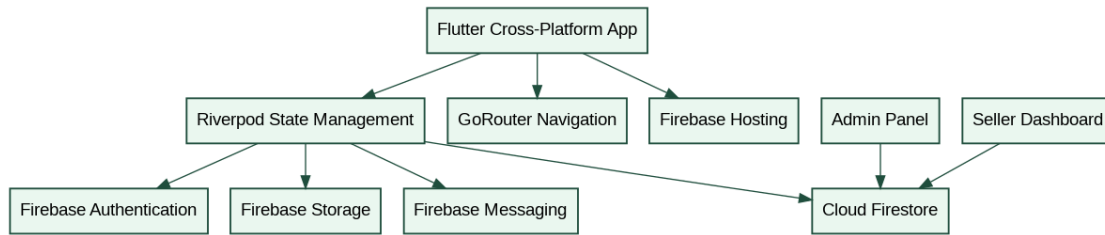
Figure A.18 - login / Account setting Screen



login / Account setting Screen

Appendix B - System Diagrams

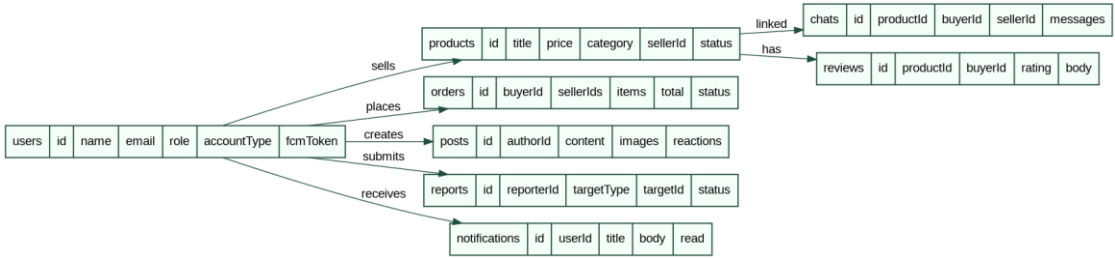
System Architecture



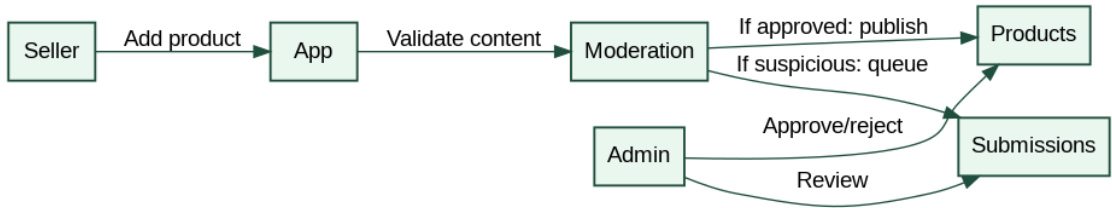
Use Case Diagram



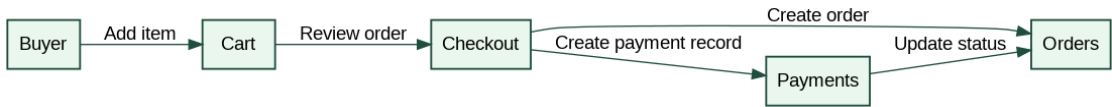
ERD

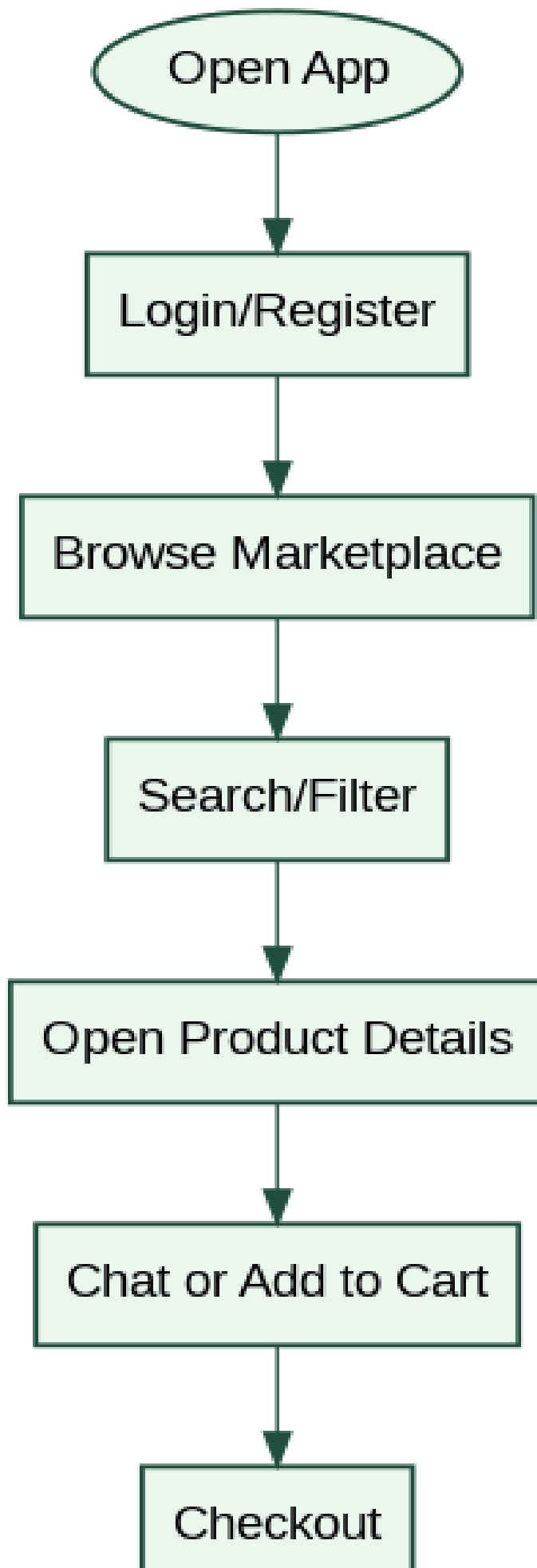


Product Moderation Sequence



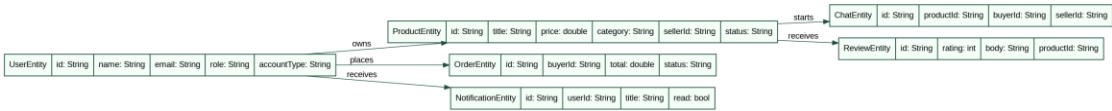
Checkout Sequence



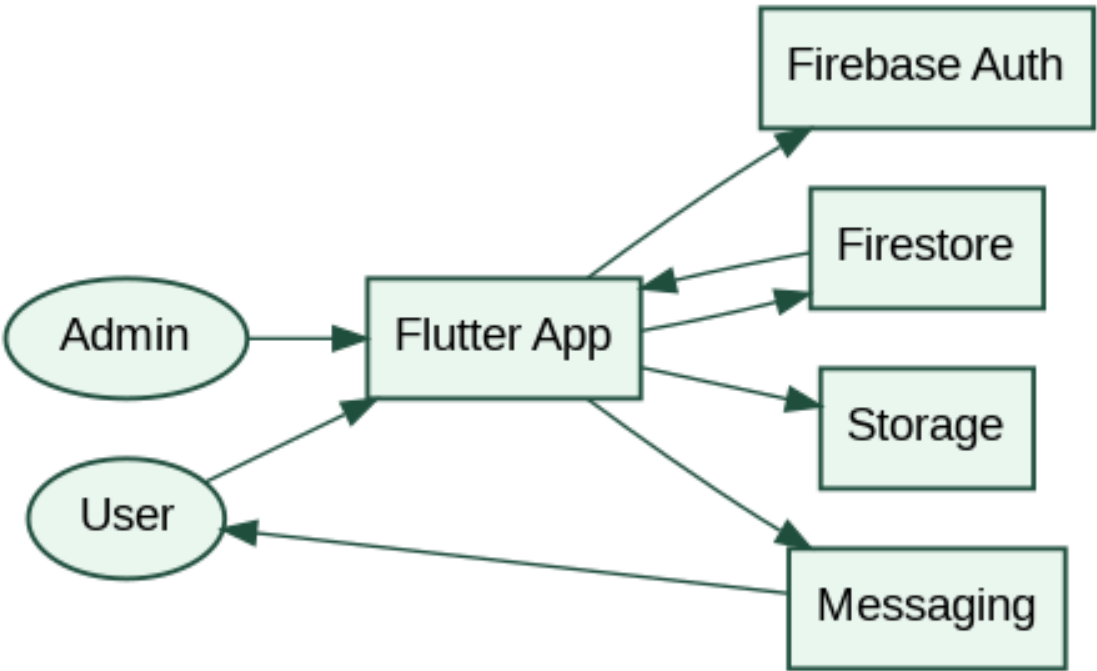


Diagra

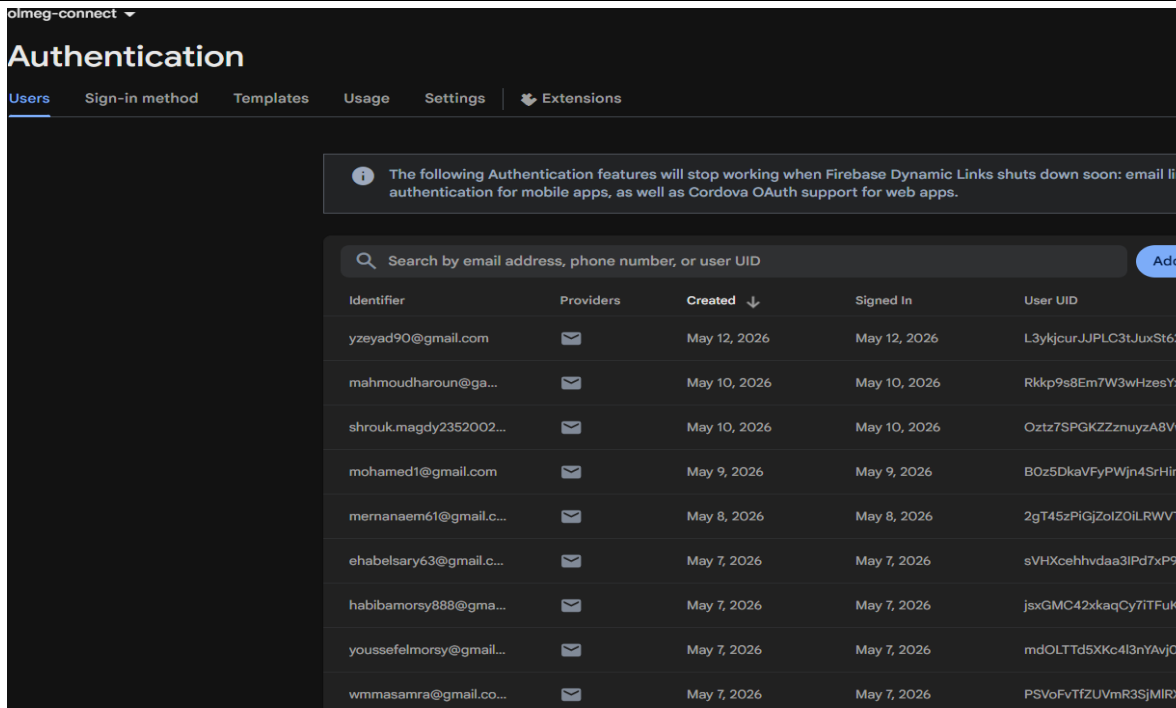
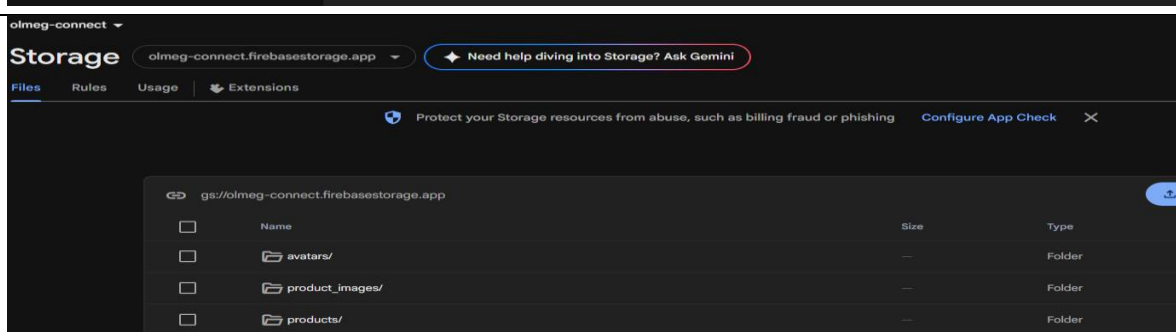
Class Diagram

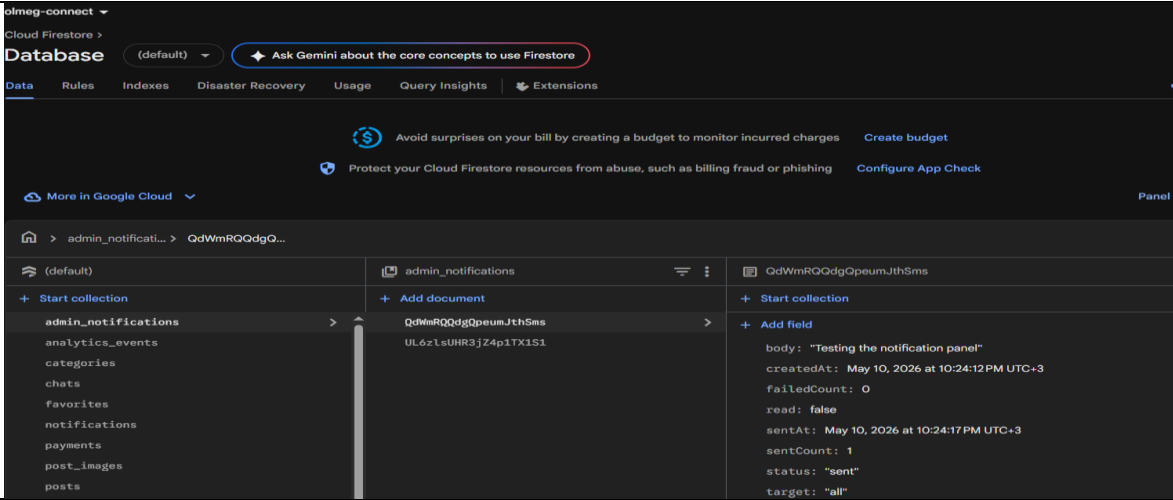


Data Flow Diagram



Appendix C - Future Links and Deployment Information

Item	Value
GitHub Repository	https://github.com/Youssef-hafez-elmorsy/OLMEG-Connect
Firebase Project ID	<i>olmeg-connect</i>
Firebase Hosting URL	https://olmeg-connect.web.app
Current App Version from pubspec.yaml	1.6.1
APK Download Link	https://drive.google.com/drive/folders/1GoLSewKVjy0WXw75xmQXQQanrdasFpgY?usp=drive_link
Figma Design Link	https://www.figma.com/design/cK4LMDMP5rVP0aLzbJVVuY?node-id=23-2
Firebase Authentication Screenshot	
Firestore Collections Screenshot	

Firestore Messaging Screenshot	
Firestore Rules Screenshot	

Deployment Commands

```
flutter pub get
flutter run
flutter test
flutter build apk --release
flutter build web --release
firebase deploy --only hosting --project olmeg-connect
firebase deploy --only firestore,storage,functions
```

References

Flutter Documentation. Google. <https://docs.flutter.dev>

Firebase Documentation. Google. <https://firebase.google.com/docs>

Riverpod Documentation. <https://riverpod.dev>

GoRouter Package Documentation. https://pub.dev/packages/go_router

Material Design Guidelines. <https://m3.material.io>

Figma Design Platform. <https://www.figma.com>

Investopedia. Online Marketplace Definition.

<https://www.investopedia.com/terms/o/onlinemarketplace.asp>